

RailCab

Task Resolution, VAPT & Improvement Documentation

Developer Task Guide

Security Fixes | VAPT Findings | Architecture Improvements

Performance | Database | Frontend | Non-VAPT Tasks

Version 1.0 | 2025 | Internal Use Only

Novelski Technologies Pvt. Ltd.

Prepared for: Development Team, VAPT Auditors, Project Stakeholders

Classification: CONFIDENTIAL

Table of Contents

SECTION A	VAPT Security Tasks	Critical / High
A1–A5	Authentication & JWT Security	Critical
A6–A11	Authorization & IDOR Fixes	Critical
A12–A17	Injection & Input Validation	Critical/High
A18–A23	Payment & Webhook Security	Critical
A24–A28	IRCTC Integration Hardening	Critical/High
A29–A33	API Security & Headers	High
A34–A38	Mobile App Security (OWASP Mobile)	High
A39–A43	Database Security	Critical/High
A44–A47	Infrastructure & Server Security	Critical/High
A48–A50	Logging & Monitoring	Medium
SECTION B	Non-VAPT Development Tasks	
B1–B5	Static Pages → Dynamic & SEO	Medium
B6–B10	Frontend UX & Feature Improvements	Medium
B11–B15	Backend Code Quality & Architecture	High
B16–B20	Performance & Latency Improvements	High
B21–B25	Database Design & Query Improvements	Medium/High
B26–B30	System Design & Scalability	High
B31–B34	DevOps & Deployment	High
APPENDIX	Summary Dashboard & Priority Matrix	

Introduction

This document is the **single source of truth** for all outstanding tasks on the RailCab platform. It is structured in two major sections:

- **Section A – VAPT Security Tasks:** Every security vulnerability, misconfiguration, and risk identified via static code analysis of the RailCab Flask backend, CORS policy, authentication layer, payment flows, database models, and infrastructure. Each task explains the exact issue, the precise fix required, and the acceptance criteria for sign-off.
- **Section B – Non-VAPT Development Tasks:** Architecture improvements, static-to-dynamic page conversions, performance and latency optimisations, database design enhancements, system design improvements, and DevOps hardening tasks.

Each task is presented as a **task card** with a unique ID, priority level (CRITICAL / HIGH / MEDIUM / LOW), the affected component, a plain-English description of the issue, exactly what needs to be done, and the acceptance criteria a developer must satisfy before closing the task.

Priority Level Legend

CRITICAL	Must be fixed before any production deployment / IRCTC go-live. Security breach risk.
HIGH	Must be fixed before VAPT audit sign-off or public launch.
MEDIUM	Should be fixed in the next sprint. User experience or stability impact.
LOW	Good to have. Improvement or polish item.

SECTION A — VAPT Security Tasks

All tasks below are directly derived from static code review of the RailCab Flask backend (irctc-final-backend-monolethic.py, 7039 lines), CORS policy, database model analysis, and architectural assessment. These are real findings, not hypothetical — line references are provided.

A1–A5 | Authentication & JWT Security

A-01	Remove Hardcoded Database Credentials from Source Code	CRITICAL
Category: Authentication / Secrets Component: Flask Backend – app.py Line 99 Effort: 2 hours		
Issue / Problem	The PostgreSQL connection string is hardcoded directly in the source file: postgresql://devuser:Mahesh%4054@13.127.220.244:5432/rail_cab. This means the live database username, password, and server IP are committed to the codebase. Anyone with access to the git repository or the .py file can access the production database directly. This is a CRITICAL breach — classified as OWASP A02:2021 Cryptographic Failures.	
What To Do	1. Immediately rotate the database password on the PostgreSQL server. 2. Remove the hardcoded DSN entirely from source code. 3. Store only in environment variable: DATABASE_URL in .env file (already partially set up). 4. Ensure .env is in .gitignore and never committed. 5. Update the fallback default to an obviously-invalid placeholder like 'postgresql://changeme'. 6. Rotate any API keys or secrets that may have been exposed in the same file.	
Acceptance Criteria	No credentials or IP addresses appear anywhere in source code. All secrets loaded exclusively via os.getenv() with no insecure defaults. .env file confirmed in .gitignore. Database password rotated.	
Code Ref	Code Reference: Line 99: DB_DSN = os.getenv('DATABASE_URL', 'postgresql://devuser:Mahesh...')	

A-02 Hardcoded Production URL and IRCTC Callback URL in Source		CRITICAL
Category: Authentication / Secrets		Component: Flask Backend – Lines 68, 94 Effort: 1 hour
Issue / Problem	Two production URLs are hardcoded as string literals: APP_BASE_URL = "https://novelskidev.online/irctc" (Line 68) and IRCTC_WS_RETURN_URL = "https://novelskidev.online/irctc/booking/return" (Line 94). This prevents running the application in staging/development environments without code changes, and exposes the production domain in source code.	
What To Do	1. Move both URLs to environment variables: APP_BASE_URL and IRCTC_RETURN_URL in .env. 2. Use os.getenv('APP_BASE_URL', 'http://localhost:5010') as the fallback for local dev. 3. Update all references in the codebase to use the env var.	
Acceptance Criteria	No production URLs hardcoded in source. URL config loaded from environment for all environments.	
Code Ref	Code Reference: Lines 68, 94	

A-03 Implement Proper JWT Authentication on All Protected Endpoints		CRITICAL
Category: Authentication		Component: Flask Backend – All 55 Routes Effort: 1 day
Issue / Problem	Code review reveals no JWT middleware or auth decorator is applied consistently across routes. Endpoints like /users/profile/<user_id>, /bookings/by-user/<user_id>, /tdr/file, and admin endpoints lack a reusable @token_required decorator. Without a consistent auth check, any unauthenticated caller can access user data. This is OWASP A07:2021 Identification and Authentication Failures.	
What To Do	1. Create a reusable @require_auth decorator that: reads Authorization header, validates JWT signature using the secret key, checks expiry, and injects user_id into request context. 2. Create a @require_admin decorator for admin-only endpoints. 3. Apply @require_auth to ALL user-specific endpoints. 4. Apply @require_admin to all /admin/* routes. 5. Verify the caller's user_id in the token matches the user_id in the URL (prevents IDOR). 6. Return 401 Unauthorized for missing/invalid tokens, 403 Forbidden for role mismatch.	
Acceptance Criteria	Every protected endpoint returns 401 without a valid JWT. Admin endpoints return 403 for non-admin tokens. All auth checks are centralised in one decorator — no duplicated auth logic.	
Code Ref	Code Reference: Global – no @token_required decorator found in codebase	

A-04 Implement Password Hashing for User Passwords		CRITICAL
Category: Authentication / Cryptography		Component: Flask Backend – users table Effort: 4 hours
Issue / Problem	The users table (Line 529) stores user passwords but there is no bcrypt or Argon2 hashing found anywhere in the codebase. Passwords appear to be stored or compared in plaintext. If the database is compromised, all user passwords are exposed immediately. This is OWASP A02:2021.	
What To Do	1. Install flask-bcrypt: pip install flask-bcrypt. 2. On registration: hash the password using bcrypt.generate_password_hash(password). 3. On login: verify using bcrypt.check_password_hash(stored_hash, provided_password). 4. Write a one-time migration script to hash all existing plaintext passwords in the database. 5. Never log or return password fields in any API response.	
Acceptance Criteria	No plaintext passwords in database. All passwords stored as bcrypt hashes. Login verified against hash.	
Code Ref	Code Reference: Line 529: users table – no hashing logic found	

A-05 Encrypt IRCTC Passwords Stored in Database		CRITICAL
Category: Authentication / Cryptography		Component: Flask Backend – user_irctc_accounts table Line 363 Effort: 6 hours
Issue / Problem	The database column irctc_user_password in the user_irctc_accounts table stores IRCTC user passwords. The developer's own comment says '# Consider encrypting this' (Line 363) — confirming passwords are stored in plaintext. IRCTC passwords give access to a government railway booking system and linked bank accounts. Exposure of this table means full IRCTC account takeover for all users. CRITICAL risk.	
What To Do	1. Use Fernet symmetric encryption (from cryptography library) to encrypt IRCTC passwords at rest. 2. Store the Fernet encryption key in environment variable IRCTC_PASSWORD_ENCRYPTION_KEY — never in code. 3. Encrypt on save: encrypted = Fernet(key).encrypt(password.encode()). 4. Decrypt on use (only at booking time): Fernet(key).decrypt(stored). 5. Write a migration to encrypt all existing plaintext passwords in the table. 6. Ensure the encryption key is never logged or exposed in API responses.	
Acceptance Criteria	All IRCTC passwords encrypted at rest using Fernet. Decryption only happens in memory at booking time. Key stored in env only.	
Code Ref	Code Reference: Line 363: irctc_user_password column – # Consider encrypting this	

A6–A11 | Authorization & IDOR Fixes

A-06 Fix IDOR: Validate User Ownership on /users/profile/		CRITICAL
Category: Authorization / IDOR		Component: Flask Backend – Line 841 Effort: 4 hours
Issue / Problem	The endpoint GET /users/profile/<user_id> takes a user_id in the URL path. If there is no check that the requesting user's JWT token matches the user_id parameter, any authenticated user can fetch any other user's profile by changing the ID in the URL. This is a classic IDOR (Insecure Direct Object Reference) — OWASP A01:2021.	
What To Do	1. In the route handler, extract caller_user_id from the validated JWT token. 2. If caller_user_id != url_user_id AND caller is not admin: return 403 Forbidden. 3. Apply the same ownership check to PATCH /users/profile, all /users//irctc-account endpoints, /tdr/by-user/, /bookings/by-user/.	
Acceptance Criteria	User A cannot access User B's profile, bookings, IRCTC accounts, or TDR data. Returns 403.	
Code Ref	Code Reference: Lines 841, 862, 991, 1075, 5855, 6192	

A-07 Restrict /booking/debug/ Endpoint		CRITICAL
Category: Authorization		Component: Flask Backend – Line 1373 Effort: 30 minutes
Issue / Problem	A debug endpoint GET /booking/debug/<client_txn_id> is publicly accessible. Debug endpoints typically expose raw database records, internal state, and implementation details. In a production IRCTC-certified system, this endpoint must not exist. It could expose booking details of any user to an unauthenticated caller.	
What To Do	1. REMOVE this endpoint entirely from production code. 2. If needed for development, protect it with an @require_admin decorator. 3. Ensure no other debug/test routes are accessible in production. 4. Add a pre-deployment checklist item to scan for debug routes before every release.	
Acceptance Criteria	GET /booking/debug/* returns 404 in production. Endpoint removed or admin-only.	
Code Ref	Code Reference: Line 1373: @app.route('/booking/debug/')	

A-08 Protect Admin Reconciliation Endpoints		CRITICAL
Category: Authorization		Component: Flask Backend – Lines 5816–5836 Effort: 2 hours
Issue / Problem	Endpoints /admin/reconciliation/start, /admin/reconciliation/stop, and /admin/reconciliation/status control a background service that matches RailCab booking records with IRCTC and Razorpay records. If these are unauthenticated, any attacker can start/stop the reconciliation service, disrupting payment tracking and causing booking data inconsistencies.	
What To Do	1. Apply @require_admin decorator to all three reconciliation endpoints. 2. Similarly protect /api/cache/clear (Line 2706) and /api/stats (Line 2682) — these expose internal metrics and allow cache manipulation.	
Acceptance Criteria	All admin endpoints return 401 without a valid admin JWT. Verified with Postman.	
Code Ref	Code Reference: Lines 5816, 5822, 5828, 2706, 2682	

A-09 Prevent Horizontal Privilege Escalation on Booking Cancellation		CRITICAL
Category: Authorization / IDOR		Component: Flask Backend – Lines 4973, 5404 Effort: 3 hours
Issue / Problem	Endpoints /booking/cancel/by-client/<client_txn_id> and /booking/cancel/by-pnr/<pnr> cancel a train ticket. If the caller's identity is not verified against the booking's owner, any authenticated user can cancel another user's ticket by guessing or obtaining a client_txn_id. This would cause serious financial and reputational damage.	
What To Do	1. Before processing cancellation, fetch the booking from DB and check booking.user_id == token_user_id. 2. If mismatch, return 403 Forbidden — do not process cancellation. 3. Apply the same check to /tdr/file, /vikalp/batch-opt, /booking/fix-payment/.	
Acceptance Criteria	User A cannot cancel User B's booking. Returns 403. Test with two separate user accounts.	
Code Ref	Code Reference: Lines 4973, 5404, 5910, 6822, 1430	

A-10 Restrict Generic IRCTC Proxy Endpoint /proxy/		CRITICAL
Category: Authorization / SSRF		Component: Flask Backend – Line 3175 Effort: 3 hours
Issue / Problem	The endpoint GET/POST /proxy/<path:rest> forwards any request to IRCTC's API servers. This is an open SSRF (Server-Side Request Forgery) gateway. An attacker can craft requests to access internal IRCTC endpoints, enumerate available services, or abuse the RailCab server's IRCTC credentials for unauthorised operations.	
What To Do	1. Implement a strict whitelist of allowed proxy paths (only the specific IRCTC endpoints the app uses). 2. Reject any path not in the whitelist with 400 Bad Request. 3. Apply @require_auth to this endpoint — anonymous access must be blocked. 4. Log all proxy requests with caller identity for audit purposes.	
Acceptance Criteria	Proxy endpoint only forwards whitelisted IRCTC paths. Returns 400 for unknown paths. Requires auth.	
Code Ref	Code Reference: Line 3175: <code>@app.route('/proxy/')</code>	

A-11 Implement Role-Based Access Control (RBAC) System		HIGH
Category: Authorization		Component: Flask Backend – Global Effort: 1 day
Issue / Problem	The application has three distinct user roles (Guest, Authenticated User, Admin/Ops) but there is no formal RBAC implementation. Role checks are likely ad-hoc or absent. Without a centralised RBAC system, it is easy for role checks to be missed on new endpoints, creating privilege escalation vulnerabilities.	
What To Do	1. Add a 'role' field to the users table: values = ['user', 'admin', 'ops_manager']. 2. Include the role in the JWT payload at login time. 3. Create @require_role('admin') and @require_role('user') decorators. 4. Document which role is required for each of the 55 endpoints.	
Acceptance Criteria	All 55 endpoints have an explicit role requirement documented and enforced. No endpoint accessible above its permitted role.	
Code Ref	Code Reference: Global – no role field in users model, no RBAC decorator found	

A12–A17 | Injection & Input Validation

A-12 Add Input Validation on All URL Parameters (SQLi / Injection Prevention)		CRITICAL
Category: Injection		Component: Flask Backend – Routes with URL params Effort: 2 days
Issue / Problem	Endpoints like /avlFareenquiry/<trainNo>/<journeyDate>/<frmStn>/<toStn> and /pnrenquiry/<pnr> accept parameters directly in the URL and pass them to IRCTC API calls and database queries. There is no validation that trainNo is numeric, journeyDate is a valid date, station codes are alphanumeric, or PNR is 10 digits. Unsanitised input is a vector for injection attacks.	
What To Do	1. Install marshmallow or use Flask's built-in request validation. 2. Define strict schemas: trainNo must match <code>r^[0-9]{4,5}\$</code> , journeyDate must be <code>YYYYMMDD</code> , station codes must be <code>r^[A-Z]{2,5}\$</code> , PNR must be <code>r^[0-9]{10}\$</code> . 3. Return 400 Bad Request with descriptive error for any validation failure. 4. Apply validation to all 55 endpoints — prioritise those with URL parameters and JSONB inputs.	
Acceptance Criteria	Invalid inputs (special chars, SQL fragments, oversized strings) are rejected with 400. SQLMap scan returns no injection findings.	
Code Ref	Code Reference: Lines 3013, 3129, 3402, 3461 – URL parameter routes	

A-13 Prevent XSS via User-Supplied Data Stored in Database		HIGH
Category: Injection / XSS		Component: Flask Backend + React Frontend Effort: 6 hours
Issue / Problem	User-supplied fields like passenger name, address, and profile fields are stored in PostgreSQL and later returned via API and rendered in the React frontend. If the frontend renders these values using <code>dangerouslySetInnerHTML</code> or similar without escaping, stored XSS is possible. An attacker could inject a script tag in their passenger name that executes in another user's browser.	
What To Do	1. Backend: strip HTML tags from all user-supplied string fields before saving to DB. Use bleach library: <code>pip install bleach</code> ; <code>bleach.clean(value, tags=[], strip=True)</code> . 2. Frontend (React): ensure no field uses <code>dangerouslySetInnerHTML</code> for user-supplied content. 3. Add Content-Security-Policy header (see task A-29). 4. Use React's default JSX rendering which escapes HTML automatically — audit for any unsafe overrides.	
Acceptance Criteria	Injecting <code>alert(1)</code> as a passenger name is stored escaped and never executes in browser. CSP header present.	
Code Ref	Code Reference: All DB write operations on user-supplied string fields	

A-14 Sanitise IRCTC API Responses Before Forwarding to Client		HIGH
Category: Injection Component: Flask Backend – IRCTC Proxy Routes Effort: 1 day		
Issue / Problem	The IRCTC proxy routes forward IRCTC API responses directly to the client without sanitisation. If IRCTC returns malformed or malicious content (or if the IRCTC endpoint is spoofed via DNS), that content could be forwarded to users. Additionally, IRCTC error responses may contain internal server details that should not be exposed to end users.	
What To Do	1. Parse IRCTC responses as JSON on the backend — never forward raw response text. 2. Extract only the required fields and return a clean, structured response. 3. Wrap IRCTC error responses into a standard error format: {error: true, message: 'Service unavailable'}. 4. Never forward IRCTC HTTP headers, internal error codes, or stack traces to the client.	
Acceptance Criteria	IRCTC error responses never reach the client verbatim. All responses are structured JSON. IRCTC internal messages are suppressed.	
Code Ref	Code Reference: Lines 3013, 3175, 3083, 3129, 3297 – IRCTC proxy routes	

A-15 Fix JSONB Injection Risk in passengerList and Booking Data		HIGH
Category: Injection Component: Flask Backend – PostgreSQL JSONB columns Effort: 6 hours		
Issue / Problem	Multiple database columns use PostgreSQL's JSONB type to store complex objects like passenger lists, booking response data, and IRCTC responses. User-controlled data flowing into JSONB fields without schema validation can lead to JSON injection, oversized payload attacks, and nested object attacks that bypass business logic (e.g. injecting a fake 'status': 'CONFIRMED' into a booking record).	
What To Do	1. Define strict JSON schemas for all JSONB inputs using jsonschema library. 2. Validate passenger_list structure before saving: each passenger must have exactly name, age, gender, berth_pref. 3. Set maximum array length (max 6 passengers). 4. Never trust JSONB data read from DB for business logic without re-validation.	
Acceptance Criteria	Oversized JSON, nested injection objects, and extra fields are rejected with 400. passengerList schema validated on every booking initiation.	
Code Ref	Code Reference: irctc_bookings table – passenger_list JSONB column	

A-16 Remove All print() Debug Statements from Production Code		HIGH
Category: Information Leakage		Component: Flask Backend – 39 print() calls Effort: 3 hours
Issue / Problem	39 print() statements exist throughout the booking cancellation and payment flows (Lines 4982, 5149, 5165, 5187, 5190, 5203, 5207, 5222, 5247, 5251, 5290, 5313, etc.). In a production environment, print() writes to stdout/stderr and is captured in server logs. These logs may contain booking IDs, PNR numbers, passenger counts, and transaction status — all sensitive data that should not appear in raw server logs accessible via VPS access.	
What To Do	1. Replace ALL print() calls with proper logger.info() or logger.debug() calls. 2. Ensure sensitive fields (PNR, user_id, payment amounts) are masked in log output: log only the first/last 4 chars of PNR, not the full number. 3. Set log level to WARNING in production — debug/info logs should not run in production.	
Acceptance Criteria	Zero print() statements in codebase (grep finds none). All logging via structured logger. Sensitive fields masked.	
Code Ref	Code Reference: 39 print() calls – Lines 4982, 5149, 5165, 5187, 5190, 5203...	

A-17 Add Request Body Size Limits and Parameter Length Checks		HIGH
Category: Injection / DoS		Component: Flask Backend – Global Effort: 2 hours
Issue / Problem	Flask does not enforce a default request body size limit. An attacker can send a multi-megabyte JSON payload to any POST endpoint, consuming server memory and causing denial-of-service. Similarly, URL parameters like station names and train numbers have no maximum length check, allowing oversized strings to be passed to IRCTC API calls and database queries.	
What To Do	1. Set Flask's MAX_CONTENT_LENGTH: app.config['MAX_CONTENT_LENGTH'] = 1 * 1024 * 1024 (1 MB). 2. Add a @app.errorhandler(413) for request too large responses. 3. In URL parameter validators, enforce: trainNo max 5 chars, station code max 5 chars, PNR exactly 10 chars, journeyDate exactly 8 chars.	
Acceptance Criteria	Requests >1MB are rejected with 413. Oversized URL params return 400. Confirmed via load testing.	
Code Ref	Code Reference: Global – app.config MAX_CONTENT_LENGTH not set	

A18–A23 | Payment & Webhook Security

A-18 Verify Payment Amount Server-Side Before Razorpay Order Creation		CRITICAL
Category: Payment Security		Component: Flask Backend – Line 1233 Effort: 4 hours
Issue / Problem	The /razorpay/create_order endpoint creates a Razorpay order with an amount received from the client request body. If the amount is not validated server-side against the actual calculated fare in the database, a user can tamper with the amount in the request (e.g., change ₹1500 to ₹1) and get a ticket booked for ₹1. This is a direct revenue loss vulnerability.	
What To Do	1. When creating a Razorpay order, NEVER use the amount from the request body. 2. Retrieve the booking's calculated fare from the database using the client_txn_id. 3. Use only the server-calculated fare to create the Razorpay order. 4. Log and alert on any discrepancy between client-provided amount and server amount.	
Acceptance Criteria	Changing the amount in the order creation request has no effect — server always uses DB fare. Tested with Burp Suite.	
Code Ref	Code Reference: Line 1233: /razorpay/create_order endpoint	

A-19 Strengthen Razorpay Webhook Signature Verification		CRITICAL
Category: Payment Security / Webhook		Component: Flask Backend – Line 1295 Effort: 4 hours
Issue / Problem	The /payment/submit endpoint receives Razorpay webhook callbacks confirming payment. Razorpay signs webhooks with HMAC-SHA256 using the webhook secret. If this signature is not verified (or verified incorrectly), a fake webhook can be sent to mark any booking as paid without actual payment. hashlib is imported (Line 41) but full webhook verification must be confirmed and hardened.	
What To Do	1. Verify the X-Razorpay-Signature header on every webhook call using: hmac.new(webhook_secret.encode(), request.data, hashlib.sha256).hexdigest(). 2. If signature does not match: log the attempt, return 400, and alert the ops team. 3. Check the razorpay_order_id in the webhook matches a known pending order in DB. 4. Mark payment as confirmed ONLY after signature verification AND order ID match. 5. Implement idempotency — process each payment_id only once (store processed payment_ids).	
Acceptance Criteria	Webhook with wrong signature returns 400. Each payment_id processed exactly once. Confirmed with Razorpay test webhooks.	
Code Ref	Code Reference: Lines 1295, 41 (hashlib import)	

A-20 Prevent Payment Replay Attacks		CRITICAL
Category: Payment Security Component: Flask Backend – Line 1295 Effort: 3 hours		
Issue / Problem	A payment replay attack occurs when a valid Razorpay payment_id and signature are captured and reused to confirm multiple bookings. If the backend does not check that each payment_id has been used exactly once, the same payment can be submitted twice to book two tickets for the price of one.	
What To Do	1. After successful payment verification, store the razorpay_payment_id in the payments table with a UNIQUE constraint (already has unique=True on transaction_id — confirm this covers Razorpay payment_id). 2. Before processing a webhook, check: does this payment_id already exist in the payments table? If yes, return 200 OK (idempotent) but do NOT process again. 3. Add a database-level unique constraint on the razorpay_payment_id column.	
Acceptance Criteria	Submitting the same Razorpay payment_id twice results in only one booking confirmation. Second submission returns 200 but creates no new booking.	
Code Ref	Code Reference: Line 597: payments table – transaction_id unique=True (verify covers Razorpay payment_id)	

A-21 Protect /booking/fix-payment/ from Unauthorised Access		HIGH
Category: Payment Security / Authorization Component: Flask Backend – Line 1430 Effort: 2 hours		
Issue / Problem	The /booking/fix-payment/<client_txn_id> endpoint appears to be an admin/operations tool to manually correct payment status. If this endpoint is accessible to regular users or unauthenticated callers, any user can self-approve a failed payment and get a booking confirmed without actual payment.	
What To Do	1. Apply @require_admin decorator — only ops team can call this endpoint. 2. Log every call with: caller admin ID, timestamp, target booking ID, old status, new status. 3. Add a reason field to the request body — ops must provide a justification. 4. Send an alert email/notification to the tech lead for every manual payment fix.	
Acceptance Criteria	Regular user JWT returns 403. Admin can call but action is logged with full audit trail.	
Code Ref	Code Reference: Line 1430: /booking/fix-payment/	

A-22 Add Rate Limiting to Payment and Booking Endpoints		HIGH
Category: Payment Security / DoS Component: Flask Backend – Payment & Booking Routes Effort: 6 hours		
Issue / Problem	There is NO rate limiting implemented anywhere in the RailCab Flask backend. Payment creation, booking initiation, and PNR status endpoints can be called at unlimited frequency. This enables: brute-force attacks on booking status endpoints, DDoS on payment flows, IRCTC API quota exhaustion (IRCTC limits API calls), and automated fare scraping.	
What To Do	1. Install Flask-Limiter: pip install Flask-Limiter. 2. Apply strict limits: /razorpay/create_order: 5/minute per user, /booking/initiate: 3/minute per user, /avlFareenquiry/*: 30/minute per IP, /pnrenquiry/*: 20/minute per IP, /validate-user: 10/minute per IP. 3. Return 429 Too Many Requests with Retry-After header. 4. Use Redis as the rate limit storage backend (Redis is already partially configured).	
Acceptance Criteria	Exceeding rate limits returns 429. IRCTC API is protected from quota exhaustion. Verified with Artillery load test.	
Code Ref	Code Reference: Lines 109-135: Redis configured but no Limiter initialised	

A-23 Validate Refund Amount Server-Side on Cancellation		HIGH
Category: Payment Security Component: Flask Backend – Cancellation Routes Effort: 4 hours		
Issue / Problem	When a user cancels a ticket, the refund amount calculation must happen server-side based on IRCTC's cancellation policy rules. If any part of the refund amount comes from client input, a user could claim a higher refund than they are entitled to.	
What To Do	1. Calculate refund amount server-side using: fare, time_to_departure, and IRCTC policy rules. 2. Never accept a refund amount from the client request body. 3. Cross-validate the refund amount returned by IRCTC API against the server-calculated amount. 4. Alert on discrepancies greater than ₹50.	
Acceptance Criteria	Modifying refund amount in the cancel request has no effect on actual refund. Server always calculates independently.	
Code Ref	Code Reference: Lines 4973, 5404 – cancellation flow	

A24–A28 | IRCTC Integration Security

A-24 Never Expose IRCTC Credentials in API Responses or Logs		CRITICAL
Category: IRCTC Security		Component: Flask Backend – Global Effort: 4 hours
Issue / Problem	IRCTC_BASIC_AUTH_USER, IRCTC_BASIC_AUTH_PASS, IRCTC_WSUSER_LOGIN, and IRCTC_WSTXN_PASS are used in API requests to IRCTC. These credentials must never appear in API responses, error messages, or log files. The wsUserTransactionPassword is injected into JSON request bodies (Lines 2853, 3019, 4194) — care must be taken that these bodies are never logged or returned.	
What To Do	1. Audit all logger.info/debug/error calls that include IRCTC request bodies — mask credentials. 2. Never include IRCTC credentials in HTTP response bodies (check all jsonify() calls). 3. Create a sanitise_irctc_body() function that strips wsUserTransactionPassword before logging. 4. Add a log filter that automatically masks any string matching IRCTC credential patterns.	
Acceptance Criteria	grep on all log files finds no IRCTC credentials. API responses contain no IRCTC auth fields.	
Code Ref	Code Reference: Lines 2853, 3019, 3721, 4194 – wsUserTransactionPassword injection points	

A-25 Validate IRCTC Booking Return Callback (/booking/return)		CRITICAL
Category: IRCTC Security / Webhook		Component: Flask Backend – Line 4777 Effort: 1 day
Issue / Problem	The /booking/return endpoint receives IRCTC's POST callback with a base64-encoded BookingResponseDTO after the user completes booking on IRCTC's WebView. If this endpoint does not validate that the callback genuinely came from IRCTC (e.g., by validating the client_txn_id against a pending booking AND verifying IRCTC's response structure), a forged callback could confirm a booking that was never paid for.	
What To Do	1. Before processing the callback: verify client_txn_id exists in DB with status PENDING. 2. Decode and validate the base64 BookingResponseDTO — check for required fields: pnr, trainNo, status. 3. Cross-check the PNR with IRCTC's PNR enquiry API to confirm the ticket is real. 4. Mark the booking as confirmed ONLY after PNR cross-verification. 5. Implement a callback nonce/token to prevent replay of old callbacks.	
Acceptance Criteria	Forged booking/return POST with a fake PNR does not result in a confirmed booking. Real PNR cross-verified against IRCTC.	
Code Ref	Code Reference: Line 4777: /booking/return – IRCTC callback handler	

A-26 Switch from IRCTC Staging to Production URLs Before Go-Live		CRITICAL
Category: IRCTC Integration		Component: Flask Backend – Lines 60, 73, 77, 83 Effort: 2 hours
Issue / Problem	All IRCTC API URLs currently point to staging: <code>stagews.irctc.co.in</code> and <code>testnget.irctc.co.in</code> . If the app goes live pointing to IRCTC staging, bookings will appear to succeed in the app but no real tickets will be issued. Users will lose money without receiving tickets.	
What To Do	1. Move all IRCTC base URLs to environment variables (<code>EXTERNAL_BASE</code> , <code>IRCTC_TATKT_BASE</code> , etc.). 2. Set staging URLs as defaults for <code>FLASK_ENV=development</code> . 3. Set production URLs in the production <code>.env</code> file. 4. Add a startup check: log a prominent <code>WARNING</code> if staging URLs are used when <code>FLASK_ENV=production</code> . 5. Create a pre-deployment checklist that verifies IRCTC env before every release.	
Acceptance Criteria	Production deployment uses production IRCTC URLs. Staging deployment uses staging URLs. Startup log clearly states which environment is active.	
Code Ref	Code Reference: Lines 60, 73, 77, 83 – IRCTC staging URLs hardcoded	

A-27 Implement IRCTC API Call Timeout and Circuit Breaker Hardening		HIGH
Category: IRCTC Resilience		Component: Flask Backend – Lines 1865–1891 Effort: 6 hours
Issue / Problem	A circuit breaker pattern is partially implemented (Lines 1865–1891) but it uses debug-level logging and may not properly handle all IRCTC failure modes. If the IRCTC API is slow or down, the <code>ThreadPoolExecutor</code> (Line 2304) spawns many waiting threads that consume server memory, potentially crashing the server under load.	
What To Do	1. Ensure the circuit breaker logs at <code>WARNING/ERROR</code> level when open — not debug. 2. Set explicit socket-level timeouts (<code>connect=5s</code> , <code>read=25s</code>) on all IRCTC requests. 3. When circuit is <code>OPEN</code> , return cached data (if available) or a clear 503 response — never hang. 4. Limit <code>ThreadPoolExecutor</code> <code>max_workers</code> to a fixed cap (max 20 total) to prevent thread exhaustion. 5. Add a Prometheus/monitoring gauge for circuit breaker state.	
Acceptance Criteria	IRCTC outage results in clean 503 responses within 5 seconds. Thread count never exceeds 25 under load testing.	
Code Ref	Code Reference: Lines 1865–1891 circuit breaker, Line 2304 <code>ThreadPoolExecutor</code>	

A-28 Secure the IRCTC WebView Bridge from Token Theft		HIGH
Category: IRCTC Security		Component: Flask Backend – Lines 4388, 4509–4551 Effort: 4 hours
Issue / Problem	The /booking/webview/<client_txn_id> endpoint renders an HTML page with debug information (Lines 4509–4551 contain CSS for a .debug-info div). The WebView bridge passes sensitive booking context to IRCTC's wsapplogin. If the client_txn_id is guessable or the WebView URL is shared, another user could hijack the booking session.	
What To Do	1. Remove the debug-info HTML div and any debug CSS from the WebView page in production. 2. Make client_txn_id non-guessable — use UUID v4 (already using uuid.uuid4() — verify this). 3. Add a one-time-use token to the WebView URL — invalidate it after first use. 4. Set a 15-minute expiry on the WebView session — reject requests after expiry.	
Acceptance Criteria	WebView URL contains no debug HTML. client_txn_id is UUID4. URL expires after 15 minutes. Second access returns 410 Gone.	
Code Ref	Code Reference: Lines 4388, 4509–4551 – WebView handler with debug-info div	

A29–A33 | API Security & HTTP Headers

A-29 Add All Required HTTP Security Headers		HIGH
Category: API Security / Headers		Component: Flask Backend – Global Response Headers Effort: 3 hours
Issue / Problem	The Flask application sets only CORS headers (Access-Control-Allow-Origin: *) on OPTIONS responses. There are NO security headers: no Content-Security-Policy, no X-Frame-Options, no X-Content-Type-Options, no Strict-Transport-Security, no X-XSS-Protection, no Referrer-Policy. Missing these headers is flagged by every VAPT tool and browser security scanner.	
What To Do	1. Install Flask-Talisman: pip install flask-talisman. 2. Configure and attach Talisman with: Strict-Transport-Security: max-age=31536000; includeSubDomains; X-Frame-Options: DENY; X-Content-Type-Options: nosniff; Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline' https://checkout.razorpay.com; Referrer-Policy: strict-origin-when-cross-origin; Permissions-Policy: geolocation=(), camera=(), microphone=(). 3. Remove Access-Control-Allow-Origin: * — replace with a specific whitelist of allowed origins.	
Acceptance Criteria	SecurityHeaders.com scan returns A+ rating. All 6 security headers present in every response.	
Code Ref	Code Reference: Lines 2730–2735: CORS headers – wildcard origin, no security headers	

A-30 Fix CORS Wildcard — Restrict to Specific Origins		HIGH
Category: API Security / CORS		Component: Flask Backend – Lines 2730–2735 Effort: 2 hours
Issue / Problem	CORS is configured with Access-Control-Allow-Origin: * meaning ANY website on the internet can make cross-origin requests to the RailCab API. This allows malicious websites to make authenticated requests on behalf of logged-in RailCab users if they have a valid JWT stored in localStorage. This is a CSRF / CORS misconfiguration vulnerability.	
What To Do	1. Replace the wildcard with a specific list of allowed origins: ['https://[user-app].vercel.app', 'https://[admin-app].vercel.app', 'http://localhost:3000']. 2. Use flask-cors library with explicit origins: CORS(app, origins=ALLOWED_ORIGINS). 3. Load the allowed origins list from an environment variable (ALLOWED_ORIGINS). 4. Ensure preflight OPTIONS requests are also validated against the allowed origins list.	
Acceptance Criteria	API requests from an unlisted origin return 403. Requests from listed origins work correctly. Tested with browser dev tools.	
Code Ref	Code Reference: Lines 2730–2735: Access-Control-Allow-Origin: *	

A-31 Add API Versioning and Deprecation Strategy		MEDIUM
Category: API Design		Component: Flask Backend – All 55 Routes Effort: 1 day
Issue / Problem	None of the 55 API endpoints use versioning (no /api/v1/ prefix). When breaking changes are introduced, both old and new mobile app users (who may have older app versions installed) will break simultaneously. This also makes VAPT re-testing harder as there is no clear version boundary.	
What To Do	1. Introduce /api/v1/ prefix for all new endpoint versions. 2. Keep existing unversioned endpoints working but mark as deprecated. 3. Add an X-API-Version response header to all responses. 4. Document the deprecation timeline.	
Acceptance Criteria	New endpoints under /api/v1/. Old endpoints return deprecation warning header.	
Code Ref	Code Reference: All routes – no versioning prefix	

A-32 Suppress Verbose Error Responses in Production		HIGH
Category: Information Leakage		Component: Flask Backend – Error Handlers Effort: 3 hours
Issue / Problem	Flask in development mode returns full Python tracebacks in HTTP responses when an unhandled exception occurs. Even with debug=False (Line 7038), if there is no custom error handler, SQLAlchemy errors and other exceptions may leak table names, query strings, and file paths to the client.	
What To Do	1. Add global error handlers for 400, 401, 403, 404, 405, 429, 500. 2. Return only generic messages in production: {'error': 'Internal server error', 'code': 500}. 3. Log full tracebacks server-side but never return them to the client. 4. Test by intentionally triggering a DB error and confirming the response shows no internal details.	
Acceptance Criteria	No stack trace, table name, or file path appears in any error response. All errors return standard JSON format.	
Code Ref	Code Reference: No @app.errorhandler(500) found in codebase	

A-33 Implement API Request/Response Audit Logging		HIGH
Category: Logging / Audit		Component: Flask Backend – Global Effort: 4 hours
Issue / Problem	There is no structured audit log recording who called which endpoint, with what parameters, and what the outcome was. For an IRCTC-certified payment application, this is mandatory. Without audit logs, it is impossible to investigate fraud, diagnose production issues, or satisfy IRCTC's compliance requirements for API access.	
What To Do	1. Add a <code>@app.before_request</code> handler that logs: timestamp, method, path, <code>user_id</code> (from JWT if present), IP address, <code>user_agent</code> . 2. Add a <code>@app.after_request</code> handler that logs: response status code, response time. 3. Store audit logs in a separate file (<code>audit.log</code>) distinct from <code>error.log</code> . 4. Ensure audit logs are rotated and retained for minimum 90 days.	
Acceptance Criteria	Every API call appears in <code>audit.log</code> with <code>user_id</code> , endpoint, status, and timestamp. Retained 90 days.	
Code Ref	Code Reference: No <code>before_request/after_request</code> audit handler found	

A34–A38 | Mobile Application Security (OWASP Mobile Top 10)

A-34 Implement SSL/TLS Certificate Pinning in Mobile Apps		CRITICAL
Category: Mobile Security		Component: Mobile App – Android & iOS Effort: 1 day
Issue / Problem	Without SSL certificate pinning, a MITM (Man-in-the-Middle) attack using a rogue CA certificate can intercept all traffic between the mobile app and the RailCab API — capturing JWT tokens, IRCTC credentials, PNR numbers, and payment data. This is OWASP Mobile M3: Insecure Communication.	
What To Do	1. For React Native: use react-native-ssl-pinning library. 2. Pin the RailCab API domain (novelskidev.online) certificate public key hash. 3. Include a backup pin for the certificate renewal. 4. Test that the app rejects requests when a proxy certificate is installed. 5. Implement a certificate rotation procedure for when the SSL cert is renewed.	
Acceptance Criteria	Proxy tool (Burp Suite) with custom CA cannot intercept API traffic from the mobile app. App shows network error and does not send data.	
Code Ref	Code Reference: Mobile app – no certificate pinning library found in project	

A-35 Secure JWT Token Storage in Mobile App		CRITICAL
Category: Mobile Security		Component: Mobile App – Android & iOS Effort: 6 hours
Issue / Problem	JWT tokens stored in React Native AsyncStorage are stored in plaintext on the device. On Android, AsyncStorage stores data in an unencrypted SQLite database. On a rooted Android device or via adb backup, all stored tokens can be extracted and used to impersonate the user. This is OWASP Mobile M2: Insecure Data Storage.	
What To Do	1. Use react-native-keychain or expo-secure-store for storing JWT tokens — these use Android Keystore and iOS Keychain (hardware-backed encryption). 2. Never store tokens in AsyncStorage, Redux state that persists to disk, or console.log output. 3. Clear the keychain entry on logout. 4. Implement short JWT expiry (15 minutes) with refresh token rotation.	
Acceptance Criteria	JWT token not found in AsyncStorage or SQLite on rooted device. Stored only in Keystore/Keychain. Verified with adb shell on Android.	
Code Ref	Code Reference: Mobile app – AsyncStorage usage for auth token storage	

A-36 Remove Hardcoded API Keys from Mobile App Build		CRITICAL
Category: Mobile Security		Component: Mobile App – React Native build config Effort: 4 hours
Issue / Problem	React Native apps bundle all JavaScript source code into a single bundle file (index.android.bundle). Any hardcoded API keys, base URLs, or Razorpay KEY_ID in the JS bundle can be extracted by reverse engineering the APK using apktool or jadx. Once extracted, attackers can call the API directly or create fraudulent Razorpay orders. This is OWASP Mobile M1: Improper Credential Usage.	
What To Do	1. Never hardcode Razorpay KEY_ID, API base URLs, or any secret in JS source. 2. For Razorpay KEY_ID (which is semi-public), ensure KEY_SECRET is NEVER in the app. 3. Use react-native-config with .env files for build-time variables — these are slightly better but still present in the bundle. 4. For truly sensitive operations, fetch tokens from the API at runtime rather than embedding in the app. 5. Enable APK code obfuscation via ProGuard/R8 for Android release builds.	
Acceptance Criteria	APK decompiled with apktool shows no API secrets. Razorpay KEY_SECRET not found in bundle.	
Code Ref	Code Reference: Mobile app – build config and Razorpay integration	

A-37 Add Root/Jailbreak Detection in Mobile App		HIGH
Category: Mobile Security		Component: Mobile App – Android & iOS Effort: 4 hours
Issue / Problem	On a rooted Android or jailbroken iOS device, security controls like Keystore protection, certificate pinning, and app sandbox can be bypassed using tools like Frida, Xposed, or objection. The RailCab app handles payment transactions and IRCTC credentials — running on a compromised device without any warning significantly increases the attack surface.	
What To Do	1. Install react-native-jail-monkey library. 2. On app startup, check JailMonkey.isJailBroken() (iOS) and JailMonkey.isRooted() (Android). 3. If rooted/jailbroken: show a warning dialog, disable payment features, and log the event server-side. 4. Consider blocking the app entirely on compromised devices for payment flows.	
Acceptance Criteria	App detects rooted Android and jailbroken iOS. Payment features are disabled on compromised devices.	
Code Ref	Code Reference: Mobile app – no root/jailbreak detection found	

A-38 Prevent Sensitive Data in Mobile App Logs and Screenshots		MEDIUM
Category: Mobile Security		Component: Mobile App – Android & iOS Effort: 4 hours
Issue / Problem	React Native apps can inadvertently log sensitive data via console.log() statements in development builds. Additionally, on Android, the recent apps switcher takes a screenshot of the app when it's backgrounded — this screenshot may capture payment screens, PNR numbers, or passenger details, stored in the device's cache accessible without authentication.	
What To Do	1. Remove all console.log() calls from production builds (use a babel plugin like babel-plugin-transform-remove-console). 2. On Android: set FLAG_SECURE on payment and booking confirmation screens to prevent screenshotting. 3. On iOS: overlay a blur screen in AppDelegate when the app is backgrounded. 4. Clear sensitive data from component state when navigating away from payment screens.	
Acceptance Criteria	No PNR, payment amount, or card details visible in recent apps screenshot. console.log absent in production APK.	
Code Ref	Code Reference: Mobile app – no FLAG_SECURE or background blur implemented	

A39–A43 | Database Security

A-39 Restrict PostgreSQL Port 5432 to VPS-Internal Access Only		CRITICAL
Category: Database Security		Component: PostgreSQL – VPS Effort: 1 hour
Issue / Problem	The PostgreSQL database is on the same VPS as the Flask API (IP: 13.127.220.244). If port 5432 is open to the public internet (which the DB connection string with the public IP suggests), the database is directly accessible from anywhere. Combined with the exposed credentials (Task A-01), this means anyone can connect to the database directly without going through the API.	
What To Do	1. In UFW: ufw deny 5432 (block external access). 2. Configure PostgreSQL's pg_hba.conf to only accept connections from 127.0.0.1 (localhost). 3. Update DB_DSN to use localhost instead of the public IP: postgresql://user:pass@localhost:5432/rail_cab. 4. Verify: nmap -p 5432 13.127.220.244 must show filtered or closed.	
Acceptance Criteria	nmap from external machine shows port 5432 closed/filtered. Only Flask API (on same server) can connect to DB.	
Code Ref	Code Reference: Line 99: DB DSN uses public IP 13.127.220.244:5432	

A-40 Apply Principle of Least Privilege to Database User		HIGH
Category: Database Security		Component: PostgreSQL – DB Effort: 2 hours
Issue / Problem	The database uses a user named 'devuser'. Dev-named database users are typically created with broad permissions during development. If this user has SUPERUSER or CREATEDB privileges, a SQL injection attack on the Flask API has full database control including reading system tables, executing OS commands (COPY TO/FROM), and creating backdoor users.	
What To Do	1. Create a dedicated application user: CREATE USER railcab_app WITH PASSWORD 'strong_random_pass'. 2. Grant only necessary permissions: GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO railcab_app. 3. REVOKE all other privileges. 4. Revoke SUPERUSER, CREATEDB, CREATEROLE from devuser. 5. Update DB_DSN to use railcab_app credentials.	
Acceptance Criteria	railcab_app user cannot DROP TABLE, CREATE USER, or read pg_shadow. Verified with psql.	
Code Ref	Code Reference: Line 99: devuser – verify privileges with \du in psql	

A-41 Enable PostgreSQL SSL/TLS Enforcement		HIGH
Category: Database Security Component: PostgreSQL – Connection config Effort: 2 hours		
Issue / Problem	The DB connection string has sslmode=require (Line 99) which is positive, but this must be verified to be actually enforced on the PostgreSQL server side. If the server has ssl=off, sslmode=require on the client only prevents the client from connecting — it does not mean the connection is actually encrypted. Data-in-transit between Flask and PostgreSQL may be unencrypted.	
What To Do	1. Confirm ssl=on in postgresql.conf. 2. In pg_hba.conf add: hostssl all all 127.0.0.1/32 scram-sha-256 (require SSL for all connections). 3. Generate a self-signed server certificate or use Let's Encrypt. 4. Test: verify SSL connection using: psql 'sslmode=require ...' and check \conninfo.	
Acceptance Criteria	psql with sslmode=disable fails to connect. All DB connections use TLS. Confirmed with pg_stat_ssl view.	
Code Ref	Code Reference: Line 99: sslmode=require in connection string – verify server-side enforcement	

A-42 Implement Database Migration Framework (Alembic)		HIGH
Category: Database Management Component: Flask Backend – Line 7030 Effort: 4 hours		
Issue / Problem	The application uses db.create_all() (Line 7030) to create database tables on startup. This approach cannot handle schema changes — if a new column is added to a model, create_all() will NOT add it to the existing table. This forces developers to manually run SQL or drop/recreate tables, risking data loss and making production deployments unsafe.	
What To Do	1. Install Flask-Migrate: pip install Flask-Migrate. 2. Initialise: flask db init. 3. Generate initial migration from current models: flask db migrate -m 'initial'. 4. Apply: flask db upgrade. 5. Replace db.create_all() with flask db upgrade in the deployment script. 6. All future schema changes must go through flask db migrate.	
Acceptance Criteria	New column added to a model is applied to live DB via flask db upgrade without data loss. No manual SQL required.	
Code Ref	Code Reference: Line 7030: db.create_all() – no migration framework	

A-43		Enable PostgreSQL Audit Logging		MEDIUM
Category: Database Security / Audit		Component: PostgreSQL – pg_log config		Effort: 3 hours
Issue / Problem	PostgreSQL's audit logging is not configured. There is no record of: which queries were run, which user ran them, when sensitive tables (users, payments, user_irctc_accounts) were accessed, or whether any suspicious bulk-read queries occurred. For IRCTC VAPT compliance, audit trails on sensitive data access are mandatory.			
What To Do	1. Install pgaudit extension: CREATE EXTENSION pgaudit. 2. Configure in postgresql.conf: pgaudit.log = 'read,write,ddl' for sensitive tables. 3. Enable log_statement = 'mod' to capture all data-modifying queries. 4. Rotate and retain DB audit logs for 90 days minimum. 5. Alert on unusual patterns: bulk reads of users table, mass deletions.			
Acceptance Criteria	pgaudit enabled. SELECT on users table creates audit log entry. Bulk queries trigger alert.			
Code Ref	Code Reference: PostgreSQL server – pgaudit not configured			

A44–A47 | Infrastructure & Server Security

A-44 Restrict SSH Access to Specific IP Addresses		CRITICAL
Category: Infrastructure Security Component: VPS – UFW / SSH config Effort: 2 hours		
Issue / Problem	Port 22 (SSH) is likely open to the public internet on the VPS (13.127.220.244). An open SSH port exposes the server to brute-force attacks 24/7. Default SSH with password authentication is one of the most common attack vectors for VPS servers. Brute-forced SSH access gives full root-level control over the database, Flask API, and all user data.	
What To Do	1. Disable password authentication: edit /etc/ssh/sshd_config — PasswordAuthentication no. 2. Enable key-only authentication: PubkeyAuthentication yes. 3. Restrict SSH to specific developer IP addresses via UFW: ufw allow from to any port 22. 4. Change SSH to a non-standard port (e.g., 2222) as an additional deterrent. 5. Install fail2ban to auto-ban IPs with repeated failed login attempts.	
Acceptance Criteria	SSH with password authentication fails. SSH from unlisted IP is blocked. fail2ban active and banning brute-force attempts.	
Code Ref	Code Reference: VPS server – SSH port 22 firewall rules	

A-45 Run Flask Behind Gunicorn WSGI Server (Not Development Server)		CRITICAL
Category: Infrastructure Security Component: VPS – Flask run config Effort: 3 hours		
Issue / Problem	The application runs via app.run() (Line 7038). Flask's built-in development server is NOT suitable for production — it is single-threaded, does not handle concurrent requests properly, and has known security issues. Running Flask's dev server in production is flagged by every VAPT scanner and violates the IRCTC API production deployment requirements.	
What To Do	1. Install Gunicorn: pip install gunicorn. 2. Create gunicorn.conf.py: workers=4, worker_class='sync', timeout=120, bind='127.0.0.1:5010'. 3. Run via systemd service: gunicorn -c gunicorn.conf.py app:app. 4. Nginx should proxy to 127.0.0.1:5010 (already configured — just change Flask binding). 5. Remove the if __name__ == '__main__': app.run() block or keep only for local dev.	
Acceptance Criteria	pgrep gunicorn shows gunicorn process. Flask dev server not running. app.run() not called in production startup.	
Code Ref	Code Reference: Line 7038: app.run(host=FLASK_RUN_HOST, port=FLASK_RUN_PORT, debug=False)	

A-46 Implement Web Application Firewall (WAF) Rules in Nginx		HIGH
Category: Infrastructure Security		Component: VPS – Nginx config
		Effort: 1 day
Issue / Problem	The Nginx reverse proxy has no WAF rules, rate limiting, or request filtering configured. All traffic — including SQL injection attempts, oversized payloads, and scanner traffic — reaches the Flask application directly. A Nginx-level WAF can block common attacks before they reach the application code.	
What To Do	1. Install ModSecurity v3 for Nginx with OWASP CRS (Core Rule Set). 2. Enable basic rules: SQLi, XSS, RFI, LFI, RCE detection. 3. Configure Nginx rate limiting: limit_req_zone for API endpoints. 4. Block common scanner user agents and malformed HTTP requests. 5. Set Nginx server_tokens off to hide version information.	
Acceptance Criteria	ModSecurity blocks basic SQLi probe: ' OR 1=1 --. Nginx headers show no server version. Rate limiting returns 429.	
Code Ref	Code Reference: Nginx config – no WAF, no rate limiting, server_tokens not set	

A-47 Separate Flask API Port from Public Internet		HIGH
Category: Infrastructure Security		Component: VPS – Firewall / Network
		Effort: 1 hour
Issue / Problem	Flask runs on port 5010. If this port is open to the public internet (not blocked by UFW), the raw Flask application is accessible directly, bypassing Nginx, SSL termination, and any WAF rules. Clients connecting directly to port 5010 get unencrypted HTTP access to all APIs.	
What To Do	1. Run Flask/Gunicorn bound to 127.0.0.1:5010 only (not 0.0.0.0). 2. Block port 5010 externally: ufw deny 5010. 3. All traffic must go through Nginx on port 443 (HTTPS). 4. Verify: curl http://13.127.220.244:5010/health must time out from external network.	
Acceptance Criteria	Port 5010 not accessible from outside the server. nmap shows port 5010 filtered. All API access via HTTPS:443 only.	
Code Ref	Code Reference: Line 62: FLASK_RUN_HOST defaults to 0.0.0.0	

A48–A50 | Logging & Monitoring

A-48 Set Correct Log Levels for Production		MEDIUM
Category: Logging		Component: Flask Backend – Logging config Effort: 2 hours
Issue / Problem	The codebase uses <code>logger.debug()</code> extensively throughout the search and availability code. In production, DEBUG-level logs generate enormous log files and may expose internal logic and IRCTC API responses. Excessive logging also impacts I/O performance.	
What To Do	1. Set log level to WARNING in production via environment variable: <code>LOG_LEVEL=WARNING</code> . 2. Reserve DEBUG level for local development only. 3. Set Flask's own logger level: <code>app.logger.setLevel(logging.WARNING)</code> . 4. Configure <code>RotatingFileHandler</code> to rotate at 50MB and keep 5 backup files.	
Acceptance Criteria	Production logs contain only WARNING and above. Log file does not grow rapidly under normal usage.	
Code Ref	Code Reference: Multiple <code>logger.debug()</code> calls – Lines 1607, 1865, 1882, 1887, 1995, 2003...	

A-49 Implement Centralised Error Alerting		MEDIUM
Category: Monitoring		Component: Flask Backend + Infrastructure Effort: 4 hours
Issue / Problem	There is no alerting when critical errors occur: failed payments, IRCTC booking failures, database connection errors, or repeated authentication failures. These issues are silently logged at best, meaning the team discovers problems only when users complain — not proactively.	
What To Do	1. Integrate Sentry (free tier) for Flask error tracking: <code>pip install sentry-sdk[flask]</code> . 2. Configure Sentry to capture all 500 errors, payment failures, and IRCTC booking failures. 3. Set up email/Slack alerts for CRITICAL events: multiple failed logins, booking failures, DB errors. 4. Create a <code>/metrics</code> endpoint (Prometheus format) for infrastructure monitoring.	
Acceptance Criteria	Every 500 error appears in Sentry within 30 seconds. Payment failures trigger Slack alert.	
Code Ref	Code Reference: No Sentry or alerting integration found	

A-50 Implement Structured JSON Logging MEDIUM	
Category: Logging Component: Flask Backend – Logging config Effort: 3 hours	
Issue / Problem	Current logging uses plain text format (RotatingFileHandler with basic formatter). Plain text logs are difficult to parse, search, and alert on. For a production system handling payments and IRCTC bookings, structured logs (JSON format) are required for log analysis tools, SIEM integration, and compliance audit trails.
What To Do	1. Install python-json-logger: pip install python-json-logger. 2. Configure formatter: CustomJsonFormatter('%(timestamp)s %(level)s %(name)s %(message)s'). 3. Every log entry should include: timestamp (ISO8601), level, user_id (masked), endpoint, request_id (UUID). 4. Ship logs to a centralised log service (e.g., Papertrail free tier or ELK stack).
Acceptance Criteria	Logs are valid JSON. Each entry has timestamp, level, user_id, and endpoint. Parseable by log analysis tools.
Code Ref	Code Reference: Lines 104–108: RotatingFileHandler with basic formatter

SECTION B — Non-VAPT Development Tasks

This section covers all non-security improvements required before and after the VAPT audit: static-to-dynamic page conversions, UX improvements, backend architecture refactoring, performance optimisations, database design improvements, and DevOps tasks.

B1–B5 | Static Pages → Dynamic & SEO

B-01	Convert Home/Landing Page to Dynamic (CMS-Driven) Page	MEDIUM
Category: Frontend – Pages Component: React Frontend – Home Page Effort: 1 day		
Issue / Problem	The RailCab home/landing page is currently static HTML — text, images, and CTAs are hardcoded in the React component. Any change to homepage content (tagline, promotional offer, featured routes) requires a code deployment. This is inefficient and means non-technical team members cannot update the homepage. SEO metadata (title, description) is also static.	
What To Do	1. Create a <code>/api/v1/content/homepage</code> endpoint in Flask that returns homepage content from a DB table or JSON config file. 2. The home page React component should fetch this content on load. 3. Add dynamic meta tags using <code>react-helmet-async</code> : title, description, <code>og:title</code> , <code>og:description</code> . 4. Add structured data (JSON-LD schema) for the business: Organization, WebSite, SearchAction types. 5. Ensure the page renders correctly with and without the API (graceful fallback to default content).	
Acceptance Criteria	Homepage content updatable without code deployment. Meta tags are dynamic. Lighthouse SEO score > 90.	
Code Ref	Code Reference: React Home component – hardcoded text content	

B-02	Convert FAQ Page to Dynamic — Admin-Editable	MEDIUM
Category: Frontend – Pages Component: React Frontend + Flask Backend Effort: 1 day		
Issue / Problem	The FAQ page is static — questions and answers are hardcoded. When users ask new common questions or answers need to be updated (e.g., changed cancellation policy), a developer must edit the component and deploy. This creates unnecessary development overhead for content changes.	
What To Do	1. Create a <code>faqs</code> table in PostgreSQL: id, question, answer, category, order, is_active. 2. Add CRUD endpoints: GET <code>/api/v1/faqs</code> (public), POST/PATCH/DELETE <code>/api/v1/admin/faqs</code> (admin). 3. Update React FAQ component to fetch from API. 4. Add an FAQ management screen to the Admin Panel. 5. Implement client-side search/filter on the FAQ page.	
Acceptance Criteria	Admin can add/edit/delete FAQs via Admin Panel without code deployment. FAQ page loads from API.	
Code Ref	Code Reference: React FAQ component – hardcoded Q&A; content	

B-03 Add Dynamic SEO Meta Tags and Sitemap to Web Portal		MEDIUM
Category: SEO / Frontend		Component: React Frontend
		Effort: 2 days
Issue / Problem	The React SPA has a single static index.html with fixed and tags. Search engines see the same metadata for every page — train search results, booking pages, and help pages all appear the same to Google. This severely limits organic search visibility which is critical for a consumer-facing railway app.	
What To Do	1. Install react-helmet-async and add page-specific meta tags to every route. 2. Generate dynamic titles: 'Book Train from Mumbai to Delhi RailCab'. 3. Create a /sitemap.xml endpoint in Flask that lists all static pages. 4. Add /robots.txt with correct Disallow rules for booking/payment pages. 5. Consider implementing SSR (Next.js) or pre-rendering for key landing pages for better SEO.	
Acceptance Criteria	Each major page has unique title and description. Sitemap.xml accessible. Google Search Console confirms pages are indexable.	
Code Ref	Code Reference: React public/index.html – single static meta tags	

B-04 Make Terms, Privacy Policy & About Pages Database-Driven		LOW
Category: Frontend – Pages		Component: React Frontend + Flask Backend
		Effort: 6 hours
Issue / Problem	Terms of Service, Privacy Policy, and About Us pages are static React components. Legal pages change frequently (IRCTC certification may require updated ToS, GST registration changes Privacy Policy). Each update requires a developer and deployment.	
What To Do	1. Create a legal_pages table: page_key (terms/privacy/about), title, content (rich text), last_updated. 2. Add GET /api/v1/pages/:page_key public endpoint. 3. Admin Panel: add a legal pages editor with a rich text editor (e.g., react-quill). 4. Cache these pages aggressively (24-hour TTL) since they change rarely.	
Acceptance Criteria	Legal team can update Privacy Policy from Admin Panel. Change reflects on website within 1 minute of cache invalidation.	
Code Ref	Code Reference: React static page components	

B-05 Implement Dynamic Station Search with Autocomplete		HIGH
Category: Frontend – UX		Component: React Frontend + Flask Backend Effort: 1 day
Issue / Problem	Train search requires typing station codes or names. Currently there is a station search API (/trainBetweenStns and boarding station endpoints) but the frontend likely shows a plain text input. Users who don't know station codes (CSTM, NDLS, etc.) cannot use the app effectively. This is the #1 UX friction point for new users.	
What To Do	1. Create a local JSON file of all Indian railway stations (name + code) — ~8000 stations. 2. Load this file on app start (bundle it with the app for offline capability). 3. Implement a fuzzy-search autocomplete component: type 'Mumbai' shows CSTM, BCT, LTT, etc. 4. Show both station name and code in the dropdown. 5. Cache the station list in Redis for web portal requests.	
Acceptance Criteria	User types 3 characters and sees matching stations within 100ms. Works offline on mobile app.	
Code Ref	Code Reference: Train search component – station input field	

B6–B10 | Frontend UX & Feature Improvements

B-06 Implement Proper Loading States and Error Handling in UI		HIGH
Category: Frontend – UX		Component: React Frontend + Mobile App Effort: 2 days
Issue / Problem	IRCTC API calls can take 5–15 seconds due to the concurrent ThreadPoolExecutor fetching availability for multiple trains. Without proper loading spinners, skeleton screens, and error states, users may think the app has frozen and tap multiple times — causing duplicate booking requests.	
What To Do	1. Add skeleton loading components (react-loading-skeleton) for train search results. 2. Show a progress indicator during booking initiation (multi-step: validating → processing → confirming). 3. Add a timeout handler: if no response in 30 seconds, show 'Taking longer than usual — please wait'. 4. Never allow the 'Book Now' button to be tapped twice — disable after first tap, re-enable on error. 5. Add global error boundary component to catch React rendering errors.	
Acceptance Criteria	Double-tap on Book Now sends only one request. Loading skeleton visible during train search. Error state shown on API failure.	
Code Ref	Code Reference: Train search and booking components – missing loading/error states	

B-07 Implement Booking Flow Progress Indicator		MEDIUM
Category: Frontend – UX		Component: React Frontend + Mobile App Effort: 1 day
Issue / Problem	The train booking flow has 6 steps (Search → Select → Passenger Details → Validate → Payment → Confirm) but there is no visual progress indicator showing the user where they are in the flow. Users frequently abandon booking flows when they cannot see how many steps remain.	
What To Do	1. Add a stepper component at the top of every booking screen: Search (1) → Select Train (2) → Passengers (3) → Payment (4) → Confirmation (5). 2. Highlight the current step and show completed steps with a checkmark. 3. On mobile, show a simplified progress bar (e.g., 'Step 3 of 5'). 4. Allow users to navigate back to previous steps (with a warning if payment was already initiated).	
Acceptance Criteria	User can see current step at all times during booking. Back navigation works without losing entered data.	
Code Ref	Code Reference: Booking flow components	

B-08 Implement Offline Mode and Network Status Handling		MEDIUM
Category: Mobile App – UX		Component: Mobile App
		Effort: 2 days
Issue / Problem	When the mobile app loses internet connectivity (common in trains passing through tunnels), API calls fail silently or show generic 'Network Error' messages. Users cannot access previously booked tickets or PNR information without connectivity.	
What To Do	1. Use @react-native-community/netinfo to detect connectivity. 2. Cache the last successful booking history and PNR status locally (AsyncStorage — non-sensitive data only). 3. Show a clear 'You are offline' banner when network is unavailable. 4. Allow viewing of already-loaded booking details and PNR status while offline. 5. Queue any API requests made while offline and retry when connection is restored.	
Acceptance Criteria	Previously loaded booking details accessible without internet. Clear offline indicator shown. Queued actions execute on reconnect.	
Code Ref	Code Reference: Mobile app – no network status handling found	

B-09 Add Push Notification Deep Linking		MEDIUM
Category: Mobile App – Feature		Component: Mobile App + Flask
		Backend
		Effort: 1 day
Issue / Problem	Push notifications (FCM) are sent for booking confirmations, driver allocation, and payment updates. However, tapping the notification likely just opens the app home screen rather than navigating directly to the relevant booking or ride tracking screen. This creates poor user experience.	
What To Do	1. Include a deep link URL in every FCM notification payload: railcab://booking/{id}, railcab://ride/{id}. 2. Configure React Navigation to handle deep links and navigate to the correct screen. 3. On Android: configure intent filters in AndroidManifest.xml. 4. On iOS: configure URL schemes in Info.plist. 5. Test: tapping a booking confirmation notification opens that booking's detail screen directly.	
Acceptance Criteria	Tapping any notification navigates to the relevant screen. Deep links work from both foreground and background states.	
Code Ref	Code Reference: FCM notification handlers in mobile app	

B-10 Implement Booking Receipt / E-Ticket Download		MEDIUM
Category: Frontend – Feature		Component: React Frontend + Mobile App + Flask Backend
		Effort: 2 days
Issue / Problem	After a train ticket is booked, users receive a PNR number but there is no PDF e-ticket or receipt download available within RailCab. Users must go to IRCTC's website to print their ticket. This breaks the 'one platform' promise of RailCab.	
What To Do	1. Create a GET /api/v1/bookings/{client_txn_id}/receipt endpoint that generates a PDF. 2. Use reportlab or weasyprint to generate a formatted PDF with: booking details, passenger list, PNR, train info, fare breakdown. 3. On mobile: use react-native-share to allow sharing/downloading the PDF. 4. Store generated PDFs temporarily in a /tmp directory (clean up after 24 hours). 5. Email the receipt PDF to the user's registered email automatically on booking confirmation.	
Acceptance Criteria	User can download or share a PDF e-ticket from My Bookings. PDF contains all booking details. Email received within 2 minutes of booking.	
Code Ref	Code Reference: Booking confirmation screen – no receipt download option	

B11–B15 | Backend Code Quality & Architecture

B-11 Break Monolithic Flask App into Blueprints		HIGH
Category: Backend Architecture Component: Flask Backend – 7039 lines, single file Effort: 3 days		
Issue / Problem	The entire backend is a single 7039-line Python file. This causes: slow import times, difficulty finding code, merge conflicts in git when multiple developers work simultaneously, inability to test modules in isolation, and risk that a bug in one section (e.g., TDR) affects the entire application at import time.	
What To Do	1. Organise into Flask Blueprints: blueprints/auth.py (user registration, login, JWT), blueprints/booking.py (all booking routes), blueprints/payment.py (Razorpay routes), blueprints/irctc.py (IRCTC proxy routes), blueprints/admin.py (admin routes), blueprints/tdr.py, blueprints/vikalp.py, blueprints/cab.py. 2. Move DB models to models/ directory. 3. Move config to config.py. 4. Create app factory function: create_app(config).	
Acceptance Criteria	No single file exceeds 500 lines. Each blueprint independently testable. pytest runs all tests.	
Code Ref	Code Reference: irctc-final-backend-monolethic.py – 7039 lines single file	

B-12 Remove asyncio Import and Fix Threading Architecture		MEDIUM
Category: Backend Architecture Component: Flask Backend – Line 42 Effort: 2 hours		
Issue / Problem	asyncio is imported (Line 42) but Flask is a synchronous WSGI application. asyncio event loops do not work correctly in a multi-threaded Flask+Gunicorn environment. Meanwhile, the code uses ThreadPoolExecutor for concurrent IRCTC calls, which is correct for sync Flask, but the unused asyncio import suggests an incomplete architecture migration that could cause confusing bugs when asyncio and threads interact.	
What To Do	1. Remove the asyncio import if it is not being used (verify first). 2. If async behavior is genuinely needed, migrate to Flask with async support (Flask 2.x + asyncio) or switch to FastAPI for the high-concurrency IRCTC proxy routes. 3. Document the threading model clearly: max thread counts, thread safety of shared state.	
Acceptance Criteria	asyncio import removed (or documented usage justified). No RuntimeError: event loop warnings in server logs.	
Code Ref	Code Reference: Line 42: import asyncio – no await/async found in codebase	

B-13 Add Comprehensive Unit and Integration Tests		HIGH
Category: Testing		Component: Flask Backend + Mobile App Effort: 1 week
Issue / Problem	No test files were found in the backend codebase. A 7039-line application handling payments and government railway bookings with zero tests means any code change can silently break critical flows (payment webhook, IRCTC booking callback) with no automated detection.	
What To Do	1. Install pytest, pytest-flask, responses (for mocking HTTP calls). 2. Write unit tests for: JWT validation, payment amount calculation, Razorpay signature verification, TDR reason validation, Vikalp eligibility logic. 3. Write integration tests for: full booking flow (mocked IRCTC + Razorpay), cancellation flow with refund calculation. 4. Target minimum 70% code coverage. 5. Add GitHub Actions CI to run tests on every pull request.	
Acceptance Criteria	pytest passes all tests. Coverage report shows >70%. CI pipeline runs on every PR.	
Code Ref	Code Reference: No test files found in project	

B-14 Implement Proper Error Response Schema Across All Endpoints		MEDIUM
Category: API Design		Component: Flask Backend – All endpoints Effort: 2 days
Issue / Problem	Error responses are inconsistent across the 55 endpoints. Some return {'error': 'message'}, others return {'message': 'error'}, some return raw IRCTC error objects. Mobile app and frontend cannot reliably parse error responses, leading to generic error messages being shown to users even when a specific actionable error message is available.	
What To Do	1. Define a standard error response schema: {'success': false, 'error': {'code': 'BOOKING_FAILED', 'message': 'Human-readable message', 'details': {}}}. 2. Create error_response(code, message, status_code, details=None) helper function. 3. Update all 55 endpoints to use this helper for error responses. 4. Create an error code catalogue: document all error codes and their meanings for the frontend team.	
Acceptance Criteria	All error responses match the standard schema. Frontend can reliably check error.code to show appropriate UI.	
Code Ref	Code Reference: Inconsistent error formats across all 55 route handlers	

B-15 Add API Documentation with Swagger/OpenAPI		MEDIUM
Category: Documentation		Component: Flask Backend
		Effort: 3 days
Issue / Problem	There is no API documentation for the 55 endpoints. The mobile app team, frontend team, and VAPT auditors must read through 7039 lines of Python to understand what each endpoint expects. This slows down integration, testing, and the VAPT audit process.	
What To Do	1. Install flasgger or flask-restx. 2. Add docstrings with Swagger annotations to all 55 endpoints. 3. Define request/response schemas for each endpoint. 4. Expose Swagger UI at /api/docs (accessible in staging, disabled in production or behind auth). 5. Generate an OpenAPI 3.0 JSON spec that can be shared with the VAPT team.	
Acceptance Criteria	GET /api/docs shows interactive Swagger UI. All 55 endpoints documented with request/response examples.	
Code Ref	Code Reference: No API documentation found in codebase	

B16–B20 | Performance & Latency Improvements

B-16	Fix Train Search Latency — Reduce P95 Response Time from 15s to <3s	HIGH
Category: Performance Component: Flask Backend – /search/trains endpoint Effort: 3 days		
Issue / Problem	The train search endpoint fetches availability for multiple trains concurrently using ThreadPoolExecutor. With 20+ trains per route and 3–4 classes each, this spawns 60–80+ HTTP requests to IRCTC simultaneously. The IRCTC API has its own rate limits and latency, causing P95 response times of 10–20 seconds. A 20-second train search makes the app feel broken.	
What To Do	1. Implement progressive loading: return train list immediately, load availability lazily via WebSocket or polling. 2. Use the existing Redis cache aggressively: cache availability data per train/date/class for 5 minutes. 3. Limit concurrent IRCTC calls to max 10 per search (not 80+). 4. Prioritise fetching the most popular classes first (SL, 3A, 2A) and lazy-load the rest. 5. Return a search_id and let the frontend poll GET /search/results/{search_id} for progressive results.	
Acceptance Criteria	Train search returns first results within 3 seconds. Full availability loads within 8 seconds. P95 < 8s confirmed by k6 load test.	
Code Ref	Code Reference: Lines 2304, 2506, 2897 – ThreadPoolExecutor spawning excessive threads	

B-17	Implement Redis Caching for Static Reference Data	HIGH
Category: Performance / Caching Component: Flask Backend – Reference data endpoints Effort: 4 hours		
Issue / Problem	Reference data like TDR reasons (/tdr/reasons), train schedules (/train/schedule/), and GST PIN lookups (/gst/pin/) are fetched from IRCTC or calculated on every request. These datasets change rarely — TDR reasons are fixed by IRCTC policy; train schedules change monthly. Repeatedly fetching them wastes IRCTC API quota and adds unnecessary latency.	
What To Do	1. Cache /tdr/reasons response in Redis with TTL=86400 (24 hours). 2. Cache /train/schedule/ with TTL=3600 (1 hour). 3. Cache /gst/pin/ with TTL=604800 (7 days). 4. Implement cache-aside pattern: check Redis first, fall back to IRCTC API, then store in Redis. 5. Add cache hit/miss metrics to /api/stats endpoint.	
Acceptance Criteria	Repeated calls to /tdr/reasons return in <10ms (from Redis). IRCTC API not called for cached responses. Cache hit rate >80% for reference data.	
Code Ref	Code Reference: Lines 6248, 3083, 6273 – no caching on reference data endpoints	

B-18 Add Database Connection Pooling and Query Optimisation		HIGH
Category: Performance / Database		Component: Flask Backend + PostgreSQL Effort: 4 hours
Issue / Problem	Each Flask request creates a new SQLAlchemy session. Without explicit connection pool configuration, SQLAlchemy creates a new database connection for each request under high load. PostgreSQL has a connection limit (default 100). Under moderate load, the API will hit 'too many connections' errors, causing 500 responses and booking failures.	
What To Do	1. Configure SQLAlchemy pool: SQLALCHEMY_POOL_SIZE=10, SQLALCHEMY_MAX_OVERFLOW=20, SQLALCHEMY_POOL_TIMEOUT=30. 2. Install and configure PgBouncer as a connection pooler between Flask and PostgreSQL. 3. Ensure all database operations use the session context manager and sessions are properly closed. 4. Add db.session.close() in a @app.teardown_appcontext handler.	
Acceptance Criteria	Under 100 concurrent users, zero 'too many connections' errors. PgBouncer dashboard shows healthy pool utilisation.	
Code Ref	Code Reference: Flask-SQLAlchemy configuration – no pool settings found	

B-19 Implement HTTP Response Compression		MEDIUM
Category: Performance		Component: Flask Backend + Nginx Effort: 2 hours
Issue / Problem	IRCTC API responses (train lists, availability data) can be large JSON payloads — often 50–200KB. These are returned without compression. For mobile users on 3G/4G networks (common for train travellers), large uncompressed responses significantly increase load time and data usage.	
What To Do	1. Enable Nginx gzip compression for application/json responses: gzip on; gzip_types application/json text/plain. 2. Install Flask-Compress as a fallback: pip install Flask-Compress. 3. Verify: curl --compressed returns a response with Content-Encoding: gzip header. 4. For train search results, also trim unnecessary fields from the IRCTC response before returning to client.	
Acceptance Criteria	API responses compressed. curl --compressed shows Content-Encoding: gzip. Train search response size reduced by >60%.	
Code Ref	Code Reference: Nginx config – no gzip compression configured	

B-20	Implement CDN Caching for Static Frontend Assets		MEDIUM
Category: Performance / Frontend		Component: Vercel + React Frontend	Effort: 4 hours
Issue / Problem	React build assets (JS bundles, CSS, images) should be served with long cache TTLs and content-based hashing. If Vercel's default caching headers are not configured, returning users re-download the entire JS bundle on every visit — wasting bandwidth and increasing page load time.		
What To Do	1. Configure Vercel.json with cache headers: Cache-Control: public, max-age=31536000, immutable for hashed assets. 2. Ensure React build generates content-hashed filenames (CRA and Next.js do this by default — verify). 3. Add a vercel.json with redirects for SPA routing (all routes return index.html). 4. Enable Vercel Edge Network caching for the API domain where safe (static reference data). 5. Use lazy loading for React route components: React.lazy() + Suspense.		
Acceptance Criteria	Lighthouse performance score >80. Returning user page load <2 seconds. Assets served with long cache TTL from CDN.		
Code Ref	Code Reference: Vercel deployment – default caching configuration		

B21–B25 | Database Design & Query Improvements

B-21 Add Composite Indexes for Common Query Patterns		HIGH
Category: Database Design		Component: PostgreSQL – Multiple tables Effort: 3 hours
Issue / Problem	The most common query on the irctc_bookings table is: WHERE user_id = ? AND irctc_booking_status = ? ORDER BY created_at DESC. Currently user_id and irctc_booking_status have individual indexes, but not a composite index covering both columns together. Without a composite index, PostgreSQL must scan all rows for a user and then filter by status — slow for users with many bookings.	
What To Do	1. Add composite index on irctc_bookings(user_id, irctc_booking_status, created_at DESC). 2. Add composite index on tdr_filings(user_id, filing_status). 3. Add composite index on payments(user_id, status, created_at). 4. Run EXPLAIN ANALYZE on the top 10 slowest queries (use pg_stat_statements). 5. Add these via Alembic migration (Task A-42).	
Acceptance Criteria	EXPLAIN ANALYZE shows Index Scan instead of Seq Scan for booking history queries. Query time < 50ms for users with 1000 bookings.	
Code Ref	Code Reference: DB models – individual indexes present, composite indexes missing	

B-22 Normalise JSONB Fields — Extract Frequently-Queried Sub-Fields		MEDIUM
Category: Database Design		Component: PostgreSQL – irctc_bookings table Effort: 1 day
Issue / Problem	The irctc_bookings table stores large JSONB blobs (irctc_response, passenger_list, booking_response). The Admin Panel likely queries these to display booking data, requiring PostgreSQL to deserialise large JSON blobs for every row. Frequently-accessed sub-fields like booking status, train number, and journey date should be promoted to proper columns for efficient querying and indexing.	
What To Do	1. Identify the top 10 most-queried fields within JSONB blobs (from application code analysis). 2. Add these as proper columns: journey_date DATE, destination_station VARCHAR(6), origin_station VARCHAR(6). 3. Populate these columns on booking creation. 4. Add indexes on the new columns. 5. Keep the JSONB blobs for full data storage but use columns for querying.	
Acceptance Criteria	Admin booking list query uses column index instead of JSONB operators. Query time reduced by >50%.	
Code Ref	Code Reference: irctc_bookings table – irctc_response, passenger_list, booking_response JSONB columns	

B-23 Implement Soft Delete Instead of Hard Delete for Bookings		MEDIUM
Category: Database Design		Component: PostgreSQL – Multiple tables Effort: 4 hours
Issue / Problem	If booking or user records are deleted from the database, payment audit trails are broken and IRCTC reconciliation cannot match historical bookings. Hard deletes of booking records are irreversible and risk compliance issues if a payment dispute arises.	
What To Do	1. Add deleted_at TIMESTAMP column to: irctc_bookings, users, tdr_filings, payments. 2. Add is_active BOOLEAN with default TRUE. 3. Replace all DELETE queries with UPDATE SET deleted_at=NOW(), is_active=FALSE. 4. Add SQLAlchemy query filters to exclude soft-deleted records by default. 5. Retain soft-deleted records for minimum 3 years (payment compliance).	
Acceptance Criteria	'Deleted' bookings not visible in API responses but present in DB with deleted_at timestamp. Recoverable by admin.	
Code Ref	Code Reference: DB models – ondelete='CASCADE' on foreign keys – risk of data loss	

B-24 Add Database-Level Constraints for Business Rules		MEDIUM
Category: Database Design		Component: PostgreSQL – Multiple tables Effort: 4 hours
Issue / Problem	Critical business rules (a booking amount must be positive, a passenger age must be between 0 and 120, a PNR must be 10 digits) are enforced only at application level. If code has a bug or is bypassed, invalid data can enter the database. Database-level constraints provide a final safety net.	
What To Do	1. Add CHECK constraints: total_fare > 0 on irctc_bookings and payments. 2. Add CHECK constraint: pnr SIMILAR TO '[0-9]{10}' (10-digit PNR). 3. Add NOT NULL constraints on all required foreign keys. 4. Add database-level ENUM types for status columns (booking_status, payment_status). 5. Deploy via Alembic migration.	
Acceptance Criteria	Inserting a negative fare or invalid PNR fails at DB level with constraint violation. Application catches and returns 400.	
Code Ref	Code Reference: DB models – minimal CHECK constraints defined	

B-25 Implement Database Backup Strategy		HIGH
Category: Database Operations		Component: PostgreSQL – VPS
		Effort: 4 hours
Issue / Problem	There is no documented database backup strategy. A single VPS running both the application and the database with no backup means: a hardware failure, accidental DROP TABLE, or ransomware attack results in permanent loss of all user data, bookings, and payment records. This is also a compliance requirement for handling financial transaction data.	
What To Do	1. Set up daily pg_dump backup: 0 2 * * * pg_dump -Fc rail_cab > /backups/rail_cab_\$(date +%Y%m%d).dump. 2. Encrypt backup files: gpg --encrypt --recipient backup.dump. 3. Upload encrypted backups to a separate S3 bucket (not the same VPS): aws s3 cp. 4. Retain daily backups for 30 days, weekly backups for 1 year. 5. Test restore procedure monthly — schedule restore drill.	
Acceptance Criteria	Daily automated backup runs without manual intervention. Restore drill succeeds within 30 minutes. Backups stored off-VPS.	
Code Ref	Code Reference: VPS cron – no backup jobs configured	

B26–B30 | System Design & Scalability

B-26 Separate Cab Booking Service from IRCTC Booking Service		HIGH
Category: System Design		Component: Backend Architecture
		Effort: 3 days
Issue / Problem	The IRCTC booking service and cab booking service are co-located in the same monolith. IRCTC API outages affect cab booking availability and vice versa. The IRCTC integration is subject to Indian government API change schedules; the cab service has completely different scaling requirements. Coupling them creates unnecessary fragility.	
What To Do	1. Extract cab booking logic into a separate Flask Blueprint initially (blueprints/cab.py). 2. Define a clear interface between the cab service and the train service: cab booking only needs: arrival_station, arrival_time, user_id — it does not need IRCTC details. 3. Long term: deploy cab service as a separate microservice on a separate port/process. 4. Use an events pattern: when a train booking is confirmed, emit a 'booking_confirmed' event that the cab service listens to.	
Acceptance Criteria	Cab booking works independently even when IRCTC API is down. Clear interface boundary between services.	
Code Ref	Code Reference: Monolithic architecture – cab and IRCTC booking mixed in same routes	

B-27 Implement Job Queue for Async Booking Notifications		MEDIUM
Category: System Design		Component: Flask Backend
		Effort: 2 days
Issue / Problem	Sending emails and SMS notifications is done synchronously within the API request handler. If the email service (SMTP/SendGrid) is slow or down, the API response is delayed and the request may time out. A failed email send should never cause a booking confirmation to fail.	
What To Do	1. Install Celery with Redis as broker: pip install celery redis. 2. Move all email sending, SMS sending, and FCM push notifications to Celery tasks. 3. API endpoint confirms booking synchronously and fires async task for notifications. 4. Configure Celery retry with exponential backoff for failed notification tasks. 5. Add Celery Flower dashboard for monitoring task queue.	
Acceptance Criteria	Booking confirmation response time <2s regardless of email service status. Notifications sent asynchronously. Failed notifications retried.	
Code Ref	Code Reference: Flask routes – synchronous notification calls in booking confirmation handlers	

B-28 Implement Health Check and Readiness Endpoints		MEDIUM
Category: System Design / Ops Component: Flask Backend Effort: 4 hours		
Issue / Problem	The existing /health endpoint (Line 2664) only returns {'status': 'ok'} and whether IRCTC credentials are set. It does not check: database connectivity, Redis availability, IRCTC API reachability, or available disk space. Load balancers and monitoring tools need a meaningful health check to route traffic away from unhealthy instances.	
What To Do	1. Enhance /health to check: DB ping (SELECT 1), Redis ping, IRCTC API reachability (HEAD request). 2. Return 200 OK only if all checks pass; return 503 if any critical check fails. 3. Add /ready endpoint: returns 200 only when the app is fully initialised and ready for traffic. 4. Add /metrics endpoint with Prometheus format: request_count, error_rate, db_query_time.	
Acceptance Criteria	/health returns 503 when DB is unreachable. Prometheus scrapes /metrics. Load balancer uses /ready for routing.	
Code Ref	Code Reference: Line 2664: /health endpoint – minimal checks only	

B-29 Add Idempotency Keys to Booking and Payment Endpoints		HIGH
Category: System Design Component: Flask Backend – Booking & Payment Effort: 4 hours		
Issue / Problem	Mobile apps on poor networks frequently retry failed HTTP requests. If a booking initiation request is retried (due to a network timeout), it could create duplicate bookings — charging the user twice and booking two tickets. Without idempotency keys, retries are unsafe.	
What To Do	1. Accept an Idempotency-Key header on /booking/initiate and /razorpay/create_order. 2. Store idempotency keys in Redis with TTL=24 hours, mapped to the response. 3. If a request arrives with an already-seen key, return the cached response immediately. 4. If a request arrives without an idempotency key on these endpoints, return 400 Bad Request. 5. Document idempotency key requirements in the API documentation.	
Acceptance Criteria	Retrying /booking/initiate with the same Idempotency-Key returns the same response without creating a duplicate booking.	
Code Ref	Code Reference: Lines 3906, 1233 – booking and payment initiation endpoints	

B-30 Design Multi-Region Deployment for High Availability		LOW
Category: System Design / Scalability		
Component: Infrastructure		Effort: 1 week
Issue / Problem	RailCab runs on a single VPS node. If this node goes down (hardware failure, network issue, or AWS availability zone outage), the entire platform is unavailable — no train booking, no cab booking, no payment. For an IRCTC-certified production service, this is unacceptable. IRCTC's API partner agreement likely requires a minimum uptime SLA.	
What To Do	1. Move to AWS ECS or a managed Kubernetes service (AWS EKS Fargate) for the Flask API. 2. Use AWS RDS PostgreSQL with Multi-AZ enabled (automatic failover). 3. Use ElastiCache Redis cluster instead of single-node Redis. 4. Put a load balancer (AWS ALB) in front of multiple Flask API instances. 5. Set up Route53 health checks and failover routing.	
Acceptance Criteria	VPS instance termination causes <30 second API downtime with automatic failover. 99.9% uptime SLA achievable.	
Code Ref	Code Reference: Single VPS node – no redundancy	

B31–B34 | DevOps & Deployment

B-31 Implement CI/CD Pipeline with Automated Security Scanning		HIGH
Category: DevOps		Component: GitHub Actions / CI
		Effort: 2 days
Issue / Problem	There is no automated CI/CD pipeline. Code is deployed manually, meaning: security issues are not caught before deployment, tests are not run automatically, dependency vulnerabilities are not scanned, and there is no consistent deployment process. Manual deployments to a government-certified system are risky and audit-unfriendly.	
What To Do	1. Set up GitHub Actions workflow: on push to main — run pytest, run bandit (Python SAST), run safety check (dependency vulnerabilities). 2. Block merge to main if any security or test checks fail. 3. On merge to main — build Docker image, run integration tests, deploy to staging. 4. Manual approval gate before deploying to production. 5. Notify Slack on deployment success/failure.	
Acceptance Criteria	Every PR runs automated tests and security scan. Insecure code cannot be merged without review. Deployment is one-click.	
Code Ref	Code Reference: No CI/CD configuration found	

B-32 Containerise Application with Docker		HIGH
Category: DevOps		Component: Flask Backend + Infrastructure
		Effort: 1 day
Issue / Problem	The Flask application runs directly on the VPS without containerisation. This means: environment differences between dev and production cause 'works on my machine' bugs, dependencies are installed globally on the VPS (no isolation), and scaling requires manual VPS configuration changes rather than simply adding container instances.	
What To Do	1. Create Dockerfile for Flask API: use python:3.10-slim base, install deps from requirements.txt, run via gunicorn. 2. Create docker-compose.yml for local development: flask api + postgresql + redis. 3. Create .dockerignore to exclude .env files, __pycache__, .git. 4. Build and push Docker image to AWS ECR in CI/CD pipeline. 5. Deploy on VPS using Docker Compose initially; migrate to ECS later.	
Acceptance Criteria	docker-compose up starts the full stack locally. Production runs from Docker container. Zero environment-specific bugs.	
Code Ref	Code Reference: No Dockerfile or docker-compose.yml found	

B-33 Set Up Staging Environment Identical to Production		HIGH
Category: DevOps		Component: Infrastructure
		Effort: 2 days
Issue / Problem	Without a staging environment, changes are tested directly in production. The IRCTC integration alone has complex staging vs production URL differences. The VAPT audit itself requires a staging/QA environment to run tests without affecting real user bookings or IRCTC's production system.	
What To Do	1. Provision a separate staging VPS or Docker environment. 2. Use IRCTC staging URLs in staging environment. 3. Use Razorpay test mode in staging. 4. Populate staging DB with anonymised test data (not real user data). 5. Deploy to staging automatically on merge to main. Deploy to production only on git tag/release.	
Acceptance Criteria	Staging URL: api-staging.railcab.in. IRCTC staging endpoints used. Razorpay test mode active. VAPT audit conducted on staging.	
Code Ref	Code Reference: No staging environment – production is the only environment	

B-34 Implement Secrets Management with Vault or AWS Secrets Manager		HIGH
Category: DevOps / Security		Component: Flask Backend + Infrastructure
		Effort: 1 day
Issue / Problem	All secrets (DB password, IRCTC credentials, Razorpay keys, JWT secret) are stored in a .env file on the VPS. If the VPS is compromised, all secrets are immediately exposed in plaintext. .env files are also frequently accidentally committed to git repositories by developers.	
What To Do	1. Migrate secrets to AWS Secrets Manager or HashiCorp Vault. 2. Application fetches secrets at startup via AWS SDK: boto3.client('secretsmanager').get_secret_value(). 3. Rotate secrets regularly: DB password monthly, JWT secret quarterly. 4. Grant the EC2/ECS IAM role only read access to the specific secrets it needs. 5. Add pre-commit hook to detect any hardcoded secrets before git commit (use detect-secrets library).	
Acceptance Criteria	No .env file on production server. Secrets fetched from AWS Secrets Manager at runtime. Accidental git commit of secrets blocked by pre-commit hook.	
Code Ref	Code Reference: .env file on VPS – all secrets in plaintext	

Appendix — Task Summary Dashboard

Metric	Count
Total Tasks	84
VAPT Security Tasks (Section A)	50
Non-VAPT Development Tasks (Section B)	34
CRITICAL Priority (block deployment)	23
HIGH Priority (required before launch)	36
MEDIUM Priority (next sprint)	23
LOW Priority (backlog)	2
VAPT CRITICAL tasks	23
VAPT HIGH tasks	21
VAPT MEDIUM tasks	6

Recommended Execution Order

Sprint 0 (Before VAPT Audit)	CRITICAL
• A-01: Remove hardcoded DB credentials • A-02: Remove hardcoded production URLs • A-39: Block PostgreSQL public access • A-44: Restrict SSH to key-only + specific IPs • A-45: Switch to Gunicorn production server • A-47: Bind Flask to 127.0.0.1 only • A-07: Remove /booking/debug endpoint	
Sprint 1 (Core Auth & Payment Security)	CRITICAL
• A-03: Implement JWT auth decorator on all routes • A-04: Add bcrypt password hashing • A-05: Encrypt IRCTC passwords (Fernet) • A-06, A-09: Fix all IDOR vulnerabilities • A-18: Server-side payment amount validation	

- A-19: Strengthen Razorpay webhook verification
- A-20: Prevent payment replay attacks
- A-34, A-35, A-36: Mobile security (SSL pinning, secure storage)

Sprint 2 (API Hardening & Infrastructure)	HIGH
• A-08, A-10, A-11: Admin endpoint protection + RBAC • A-12, A-13: Input validation + XSS prevention • A-22: Rate limiting (Flask-Limiter) • A-29, A-30: Security headers + CORS fix • A-40, A-41: DB user privileges + SSL • A-42: Alembic database migrations • A-46: Nginx WAF (ModSecurity + OWASP CRS)	
Sprint 3 (Non-VAPT: Performance & UX)	MEDIUM
• B-16: Fix train search latency • B-11: Break monolith into Blueprints • B-05, B-06: Station autocomplete + loading states • B-17, B-18: Redis caching + DB connection pooling • B-21: Composite database indexes • B-25: Database backup strategy • B-31, B-32: CI/CD + Docker containerisation	
Sprint 4 (Features & Scalability)	LOW
• B-01–B-04: Dynamic pages (Home, FAQ, Legal, SEO) • B-07–B-10: UX improvements (stepper, offline, notifications, receipt) • B-13, B-15: Tests + API documentation (Swagger) • B-26–B-30: System design improvements • B-33, B-34: Staging env + secrets management	

This document reflects findings from static code analysis of irctc-final-backend-monolethic.py (7039 lines), architectural review, database model analysis, and infrastructure assessment of the RailCab platform. All VAPT tasks (Section A) must be completed and verified before submitting to C-DAC for IRCTC VAPT certification. Generated by: Novelski Technologies Engineering Team | Version 1.0 | 2025 | CONFIDENTIAL